

Михаил Разаков

Compiler Engineer · LLVM · Language Infrastructure

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Инженер по компиляторам с профессиональным опытом LLVM и статического анализа. В МЦСТ работал с LLVM 22: loop/interprocedural analysis, legality и IR transformations; в ИСП РАН участвую в разработке SharpChecker — промышленной платформы статического анализа C#/ .NET. В независимых проектах развиваю консервативные LLVM-преобразования, Roslyn fixed-point data-flow и UniversalToolchain — compiler/language infrastructure с typed artifact contracts и детерминированным planning; доклад по этой архитектуре принят на LangDev'26.

Professional compiler / analysis

МЦСТ: LLVM passes, loop/interprocedural analysis и legality; ИСП РАН: SharpChecker, статический анализ C#/ .NET.

Compiler architecture

UniversalToolchain: typed artifacts; dependencies/conflicts, provider selection, pass order, artifact routes и backends сводятся в immutable LanguagePlan до исполнения.

Verification & backends

Conservative rejection paths, LLVM Verifier и before/after execution; interpreter/CIL parity; отдельный x86-64 codegen/register-allocation pipeline.

ОПЫТ

2026 — сейчас

ИСП РАН — инженер по статическому анализу

- Участвую в разработке SharpChecker — промышленной платформы статического анализа C#/ .NET в ИСП РАН.

июль — август 2026

МЦСТ — стажёр по разработке компиляторов

- Реализовал LLVM LICM-pass с loop analysis, проверками side effects/ speculative safety и hoisting loop-invariant инструкций в preheader.
- Разработал консервативный прототип межпроцедурного global-IV pass: APInt affine evolution, транзитивные эффекты вызовов, отдельная legality-фаза и LLVM IR transformation; неподдерживаемые CFG/call cases отклоняются.

ПРОЕКТЫ

LLVM Interprocedural Global IV Optimization

Консервативный LLVM 22 prototype/MVP с реальным IR transform; 29 positive/negative regression cases проверяют verifier validity, idempotence и before/after execution.

UniversalToolchain

LanguageRuntime проверяет exact planned package/component graph и создаёт выбранные компоненты без повторного планирования; контракт защищён determinism/exact-binding checks, ownership guards и interpreter/CIL parity.

DerefAfterNullAnalyzer

Диагностика потенциально небезопасных dereference paths поверх реального CFG, включая повторную обработку successors до стабилизации состояния.

x86-64 Codegen & Register Allocation Lab

Reference interpreter и differential execution проверяют семантическую эквивалентность сгенерированного кода; allocator contract валидируется отдельно.

КОМПЕТЕНЦИИ

Compilers

C++, LLVM 22, LLVM IR, optimization passes, interprocedural analysis, CFG/SSA, dominators, loop analysis, LICM, legality analysis

Static / Program Analysis

C#, .NET, Roslyn ControlFlowGraph, fixed-point data-flow, state propagation and joins

Compiler Infrastructure

typed artifact contracts, IR/lowering pipelines, deterministic planning/pass ordering, backend/runtime composition, executable verification

Codegen & Systems

Rust, SysV x86-64, liveness/interference, register allocation, Linux, CMake

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия, 2026–2030

ДОСТИЖЕНИЯ

Доклад по архитектуре UniversalToolchain принят на LangDev'26; конференция пройдёт 8–9 октября 2026 в Малаге.

Москва / удалённо